

第60回技能五輪全国大会
「電子機器組立て」職種

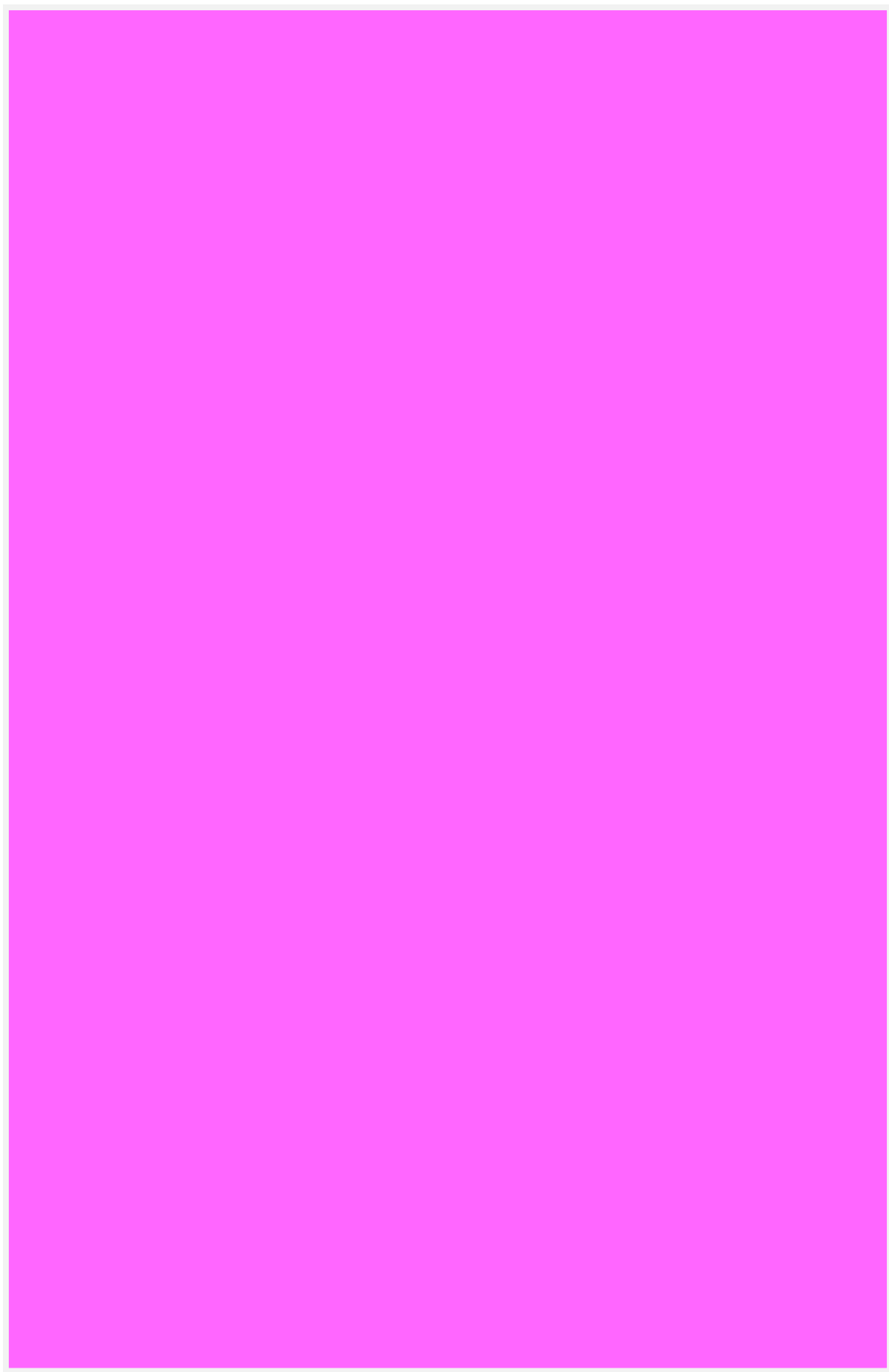
競技Ⅲ

課題仕様書

2022年11月4日
競技時間 2時間00分

選手番号：

選手氏名：



1 機器の概要

1.1 まえがき

現在，IoT：Internet of Things（モノのインターネット）という言葉が広く普及しています。IoT とは，ネットワークに接続されたデバイスが，人手を介さずにデータを送受信する仕組みのことです。さまざまなデバイスから大量のデータを収集することができ，それらをビッグデータとして分析・フィードバックするといった使われ方が拡大しています。一方で，ネットワークに接続されたデバイスのセキュリティが不十分なため，盗聴や改ざんなどセキュリティリスクが拡大し問題となっています。IoT 機器はカメラ画像などプライバシーを含む情報を外部とやり取りし，クラウドを活用することもあるため，セキュリティ対策は必須となっています。

これらの問題を解決する技術が，情報セキュリティです。情報セキュリティにはさまざまな技術がありますが，その一つに「暗号」という技術があります。暗号とは，秘密にしたい情報を鍵を使って意味不明の暗号文に変換し，正規の鍵を持っているものだけが元の情報に復号できる技術です。図 1.1 に暗号の例としてシーザー暗号を示します。

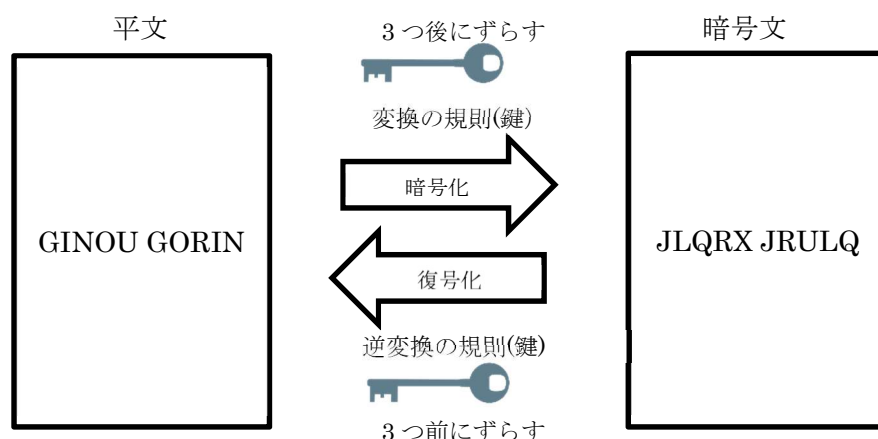


図 1.1 暗号の例

しかし，その暗号文または鍵（以下，暗号文とする）をなくしてしまうと正規の受信者でも復号できなくなり信頼性が低下します。また，その暗号文をなくさないために複製を作ると，盗難の危険性が高まり安全性が低下します。この暗号文が多くても少なくても不都合があるという相反する要求に対して，安全性と信頼性を両立する情報セキュリティ技術として「秘密分散」があります。

秘密分散とは，情報を意味不明のいくつかの情報に分割し（分散情報），決められた数の分散情報が集まると元の情報に復号できる技術です。よって，決められた数未満であれば，たとえ分散情報を失ったとしても情報漏洩を防ぐことができ，逆に決められた数の分散情報さえ集めれば，情報を復号することができます。

競技Ⅲでは，IoT と情報セキュリティをテーマとした課題とします。課題で用いる IoT ボードには，Wi-Fi 機能内蔵のマイコン ESP32 を搭載しています。このマイコンを介してネットワークに接続します。ネットワークに接続した ESP32 マイコンは，サーバにアクセスすることができ，双方向にデータのやり取りが可能になります。今回

の課題ではサーバを制御することはありません。みなさん一人ひとりは IoT 機器側の制御を行います。

1.2 関連技術

(1) 視覚復号型秘密分散

秘密分散は、秘密情報を n 個の分散情報（暗号文）に分割します。そのうち k 個以上の分散情報を集めると元の情報を復元することができます。 $k-1$ 個の分散情報では、情報を全く復元することができません。このような手法を (k,n) しきい値法とよびます。ただし、 $k \leq n$ となります。

(2,2)しきい値法による視覚復号型秘密分散の例を図 1.2 に示します。2 枚の透明シート A と B に意味の分からない画像（暗号文）が描かれています。その 2 枚のシートを重ねるという復号化処理により秘密情報が現れ（復号）内容を見ることができます。

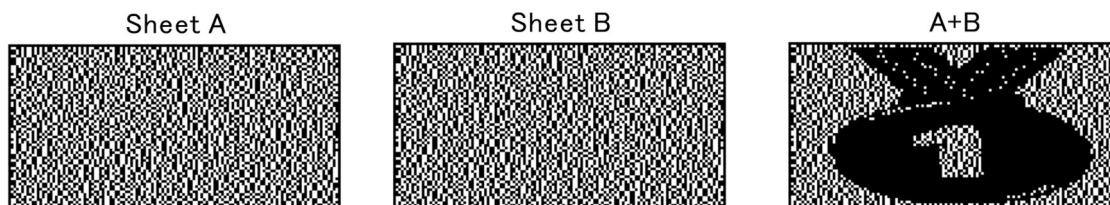


図 1.2 (2,2) しきい値法による視覚復号型秘密分散の例

2 枚の分散画像を重ねた復号例を図 1.3 に示します。A と B の透明シートを重ね合わせると、左上が濃いという情報が現れていることが分かります。A+B を濃くする部分のパターン例と濃くしない部分のパターン例を図 1.4 に示します。

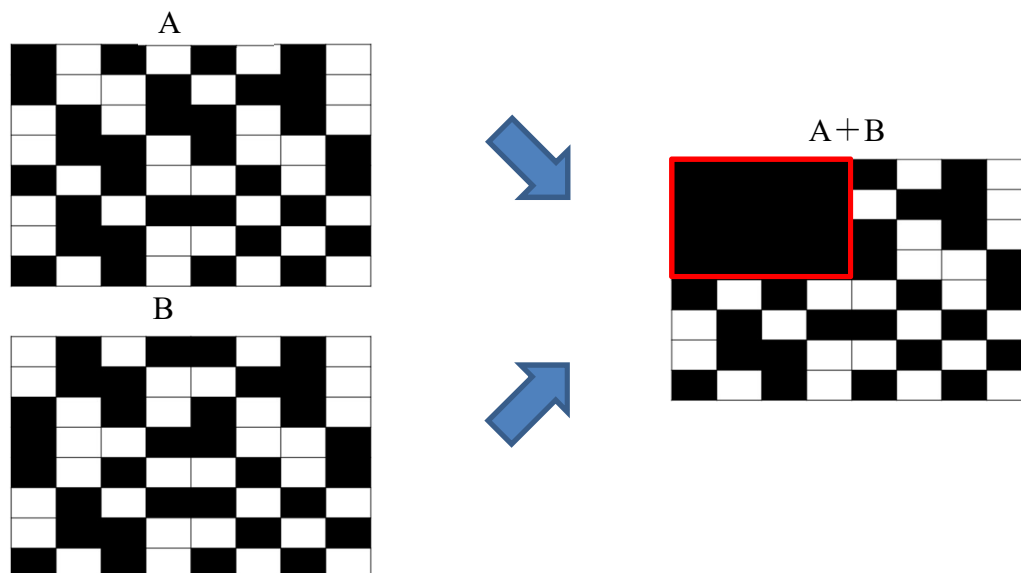


図 1.3 秘密画像の復号の例

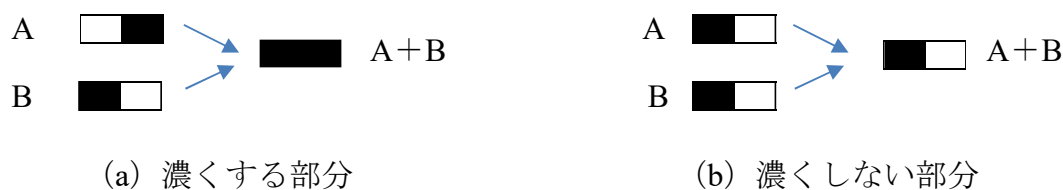


図 1.4 A+B のパターン例

(2) XBM (X Bit Map)

今回の課題では、OLED ディスプレイに XBM フォーマットで画像を表示します。XBM は、UNIX 系の GUI システムである X Window のビットマップ画像のための二値（白黒）画像ファイルフォーマットです。

ビットマップ画像とは、画素（ドット）と呼ばれる点の集まりで構成される画像データ形式の一つです。画素の 1 ドットを 1 ビットに対応させ、8 ドット（1 バイト）単位で 16 進数表記として配列に記述します。図 1.5 に 32×32 ドット画像の配列の割当例を示します。画像左上 1 行目の 8 ドットの画素情報を 16 進数表記で配列の最初の要素に格納します。ただし、このとき 8 ドットの左端が LSB に、右端が MSB になります。その後、1 バイトを単位として、16 進数表記で配列の各要素に 8 ドットの情報を一次元配列として格納します。図 1.6 に 32×32 ドット画像の XBM フォーマットの例を示します。先頭の `#define` 文は画像の横と縦の画素サイズを表しています。二次元画像を一次元配列として扱います。

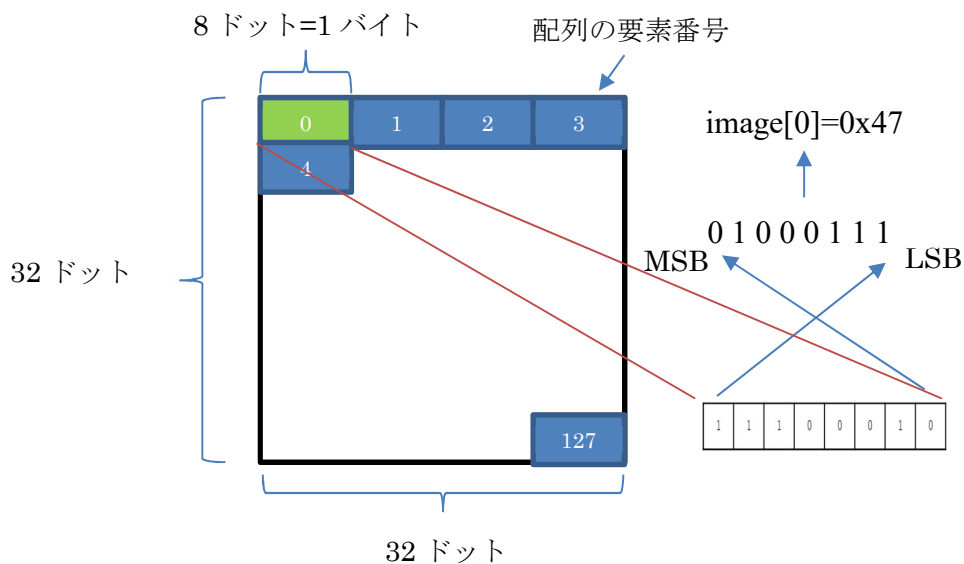


図 1.5 32×32 ドット画像の配列割当例

```
#define image_width 32  
#define image_height 32  
static char image[128] = { 0x47, 0x4a, 0xff, ..., 0x25, 0x00 };  
                                └──────────────────┘  
                                128 個
```

図 1.6 32×32 ドット画像の XBM フォーマットの例

(3) MQTT プロトコル

今回使用する ESP32 マイコンは Wi-Fi 機能を内蔵しており, ESP32 マイコンが起動すると Wi-Fi ルータに自動的に接続されます. IP は DHCP で振り分けられますが, IP で機器を判別するわけではありません. 特殊なプロトコル「MQTT」によって区別しています.

「MQTT」は 1999 年に IBM 社と Eurotech 社により考案されたプロトコルです. MQTT は Pub/Sub 型通信と呼ばれる種類の通信モデルであり, 図 1.7 に MQTT のしくみを示します.

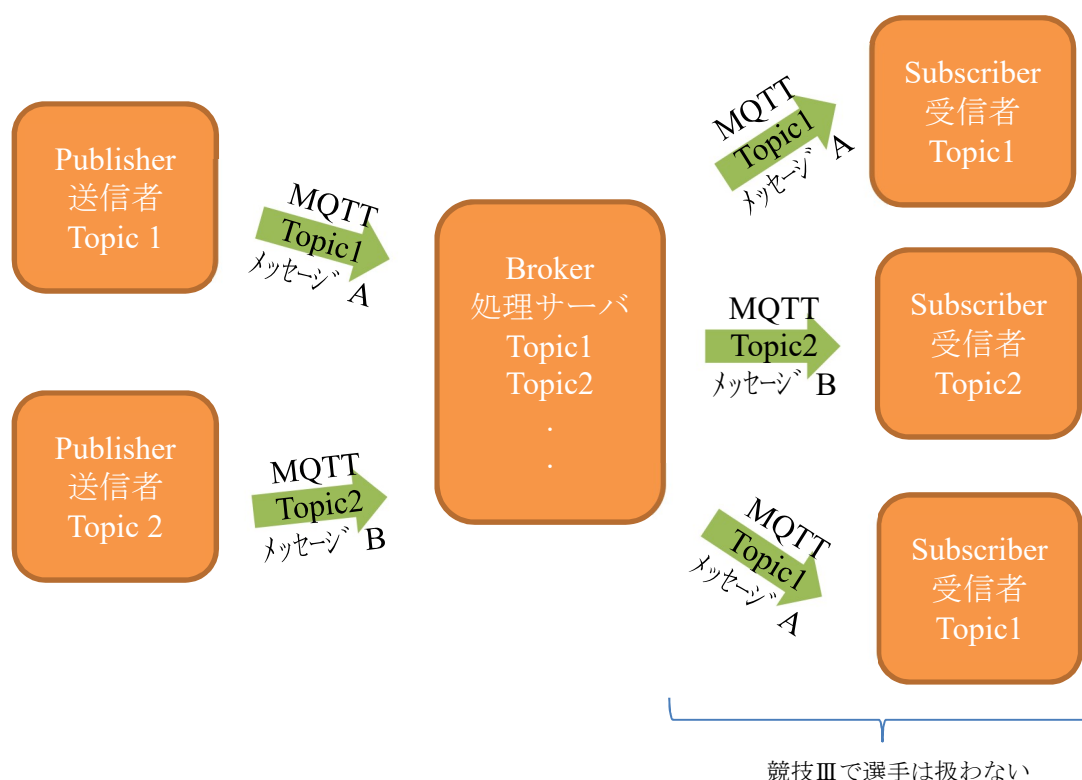


図 1.7 MQTT のしくみ

Publisher (送信者), Broker (処理サーバ), Subscriber (受信者) の三つの役割があります. Publisher はメッセージを Broker に送ります. Broker はそれを処理してメッセージを Subscriber に送信します. 例えば, 温度データを Publisher が送る場合の“実際の温度データ”がメッセージにあたります.

図 1.7 のように Publisher が話題 (Topic) を設定し, メッセージが Broker に送られ

ます。Broker は、その話題（Topic）を設定しているすべての Subscriber（購読という意味もある）に届くメッセージを送ります。MQTT プロトコルは、Topic とメッセージで構成されています。メッセージとして、数値や文字などが扱えます。

Topic は図 1.8 のように “/” で区切られた階層化されたデータをもっています。図の例では各選手ごとの “Topic” を設定しています。

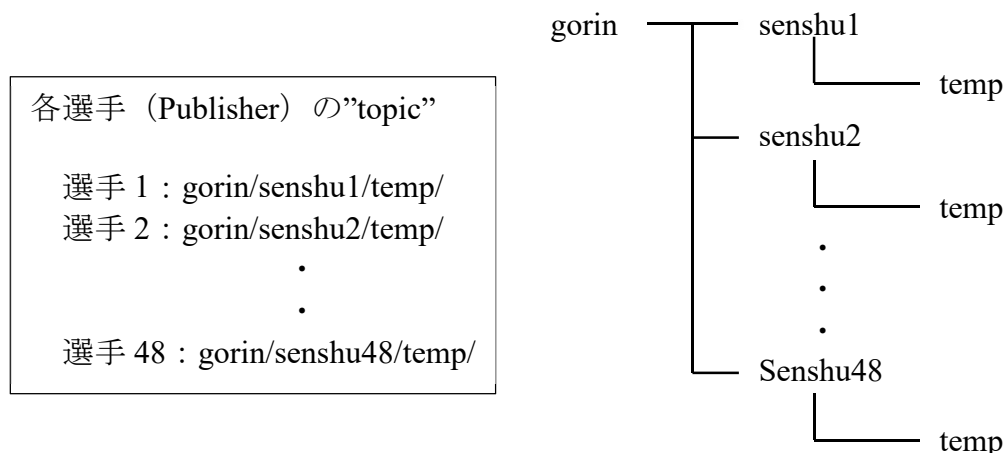


図 1.8 Topic の階層構造例

“gorin/senshu1/temp” という Topic では、“gorin” の階層の下に “senshu1” という選手の層があり、さらに各選手には “temp” 層があります。こういう階層構造は PC でいうところのフォルダの PATH のようなものを思い浮かべると分かりやすいかもしれません。

今大会の競技Ⅲで、各選手は、Publisher（送信者）として MQTT プロトコルを用いて決められた Topic にメッセージを付けて送信することが課題として含まれています。なお、サーバからのメッセージの受信については、ESP32 マイコンにあらかじめ必要な処理が記述されています。このため、課題ではプログラムの記述は必要ありません。

1.3 機器の構成

（1）構成機器

プログラム設計課題で使用する電子機器は、「IoT ボード」です。図 1.9 に機器のブロック図を、表 1.1 に PIC との信号割付表を示します。IoT ボードの構成機器は、以下のとおりです。

- IoT ボード
- マルチファンクションボード（選手持込み）
- カップリングボード（選手持込み）
- AC アダプタ（DC 9V, 1.3 A）（選手持込み）

マルチファンクションボードと「IoT ボード」は、カップリングボードを介して接続します。

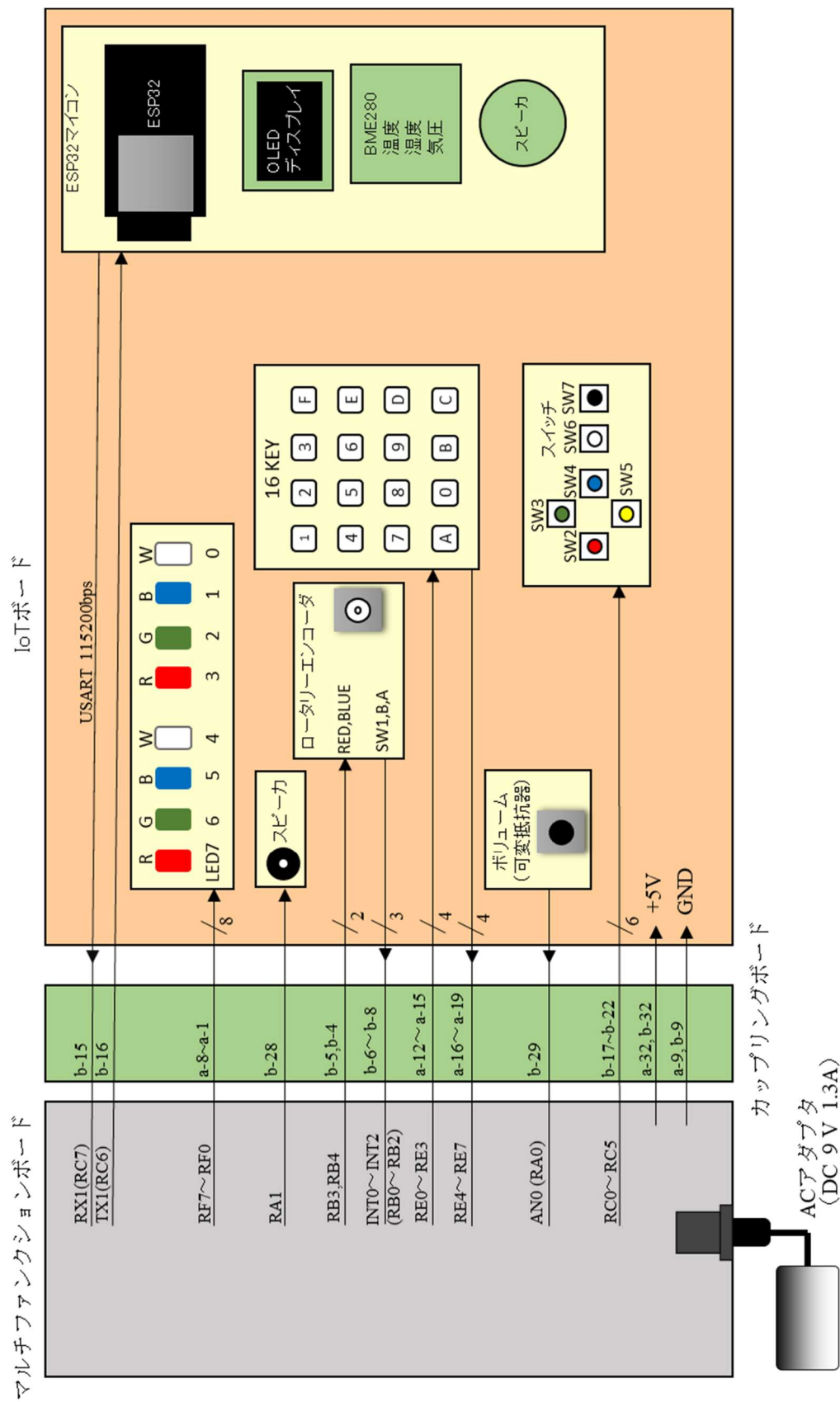


図 1.9 IoTボードのブロック図

表 1.1 IoT ボードの信号割付け表

端子番号	PIC18F6722 端子名	信 号 名
b-6	INT2(RB2)	ロータリスイッチ SW1
b-22	RC0	タクトスイッチ SW2
b-21	RC1	タクトスイッチ SW3
b-20	RC2	タクトスイッチ SW4
b-19	RC3	タクトスイッチ SW5
b-18	RC4	タクトスイッチ SW6
b-17	RC5	タクトスイッチ SW7
b-8	INT0(RB0)	ロータリスイッチ A
b-7	INT1(RB1)	ロータリスイッチ B
a-8	RF7	LED7 RED
a-7	RF6	LED6 GREEN
a-6	RF5	LED5 BLUE
a-5	RF4	LED4 WHITE
a-4	RF3	LED3 RED
a-3	RF2	LED2 GREEN
a-2	RF1	LED1 BLUE
a-1	RF0	LED0 WHITE
b-5	RB3	ロータリスイッチ RED
b-4	RB4	ロータリスイッチ BLUE
b-28	RA1	スピーカ
b-29	AN0(RA0)	ボリューム
b-15	RX1(RC7)	USART 受信
b-16	TX1(RC6)	USART 送信
a-12	RE0	16 キーパッド Y1
a-13	RE1	16 キーパッド Y2
a-14	RE2	16 キーパッド Y3
a-15	RE3	16 キーパッド Y4
a-16	RE4	16 キーパッド X1
a-17	RE5	16 キーパッド X2
a-18	RE6	16 キーパッド X3
a-19	RE7	16 キーパッド X4
a-32	+5 V	+5 V
b-32	+5 V	+5 V
a-9	GND	GND
b-9	GND	GND

(2) IoT ボードの動作概要

IoT ボードには, ESP32 マイコンが搭載されており, Wi-Fi 接続, センサ (BME280), OLED ディスプレイ, スピーカの制御に使用しています. それ以外の LED0~7, スピーカ, ロータリエンコーダ, ボリューム (可変抵抗器) VR1, スイッチ SW2~SW7, 16 キーパッド KEY1 は, マルチファンクションボードから直接制御を行います.

ESP32 マイコンは, Wi-Fi 接続をおこない, ネットワークにアクセスができます. ネットワーク上で, MQTT を用い Publisher, Subscriber となりデータのやり取りを行います. また, センサ (BME280) では温度, 湿度, 気圧が測定できます. OLED ディスプレイは, 128×64 ドットの有機 EL グラフィックディスプレイで, 1 行に 13 文字, 4 行表示することができます.

マルチファンクションボードの PIC と ESP32 マイコンは USART で接続しています. ただし, MQTT やセンサ, ディスプレイは専用の関数でやり取りを行うため, USART を意識する必要はありません.

LED0~7 は, LED7 と LED3 が赤 (R), LED6 と LED2 が緑 (G), LED5 と LED1 が青 (B), LED4 と LED0 が白 (W) になっています. スイッチは SW2~SW7 まであり, 某ゲームコントローラ的な配置になっており, それぞれ色が決まっています. ロータリエンコーダの出力は A 相と B 相で, 押しボタンスイッチ SW1 も有しています. また, 赤と青の LED も搭載しています. ロータリエンコーダは, モードの切り替えなどに使用します.

16 キーパッドは, マトリックス状にスイッチが配置されており, 4 列の入力 X1~X4, 4 行の出力 Y1~Y4 が接続されています. スピーカは, 電圧の High と Low を繰り返すことによって, その周波数の音が鳴ります. ボリューム (可変抵抗器) は, 0 V から 5 V までの電圧を出力し, アナログ入力 AN0 に入力されます.

競技Ⅲでは, IoT ボードを用いたプログラミング課題とした競技を実施します.

(3) IoT ボードの動作確認

サーバで配布した “iot_check.hex” を PIC18F6722 に書き込むことで IoT ボードの動作確認ができます. 電源をオンにしてから “ピピッ” とブザーが鳴って Wi-Fi に接続してから使用して下さい.

hex ファイルを書き込むときの注意

- ・ MPLAB X IPE で書き込むとき, MPLAB X IDE は終了してください.
- ・ 書き込み後は, ICD, PIC Kit などのライターは MF ボードから外してください.

動作確認方法を表 1.2 に示します.

表 1.2 IoT ボードの動作確認

通常のチェックモード		
部品	方法	動作
ロータリエンコーダ	回転	OLED ディスプレイの 1 行目に現在のモードが表示されています モードの切り替え (mode1～mode6 がループ) 時計回り → mode が進む 反時計回り → mode が戻る
SW1	押下	ロータリエンコーダの LED 押下前 赤 → 押下後 青 押下している間 ブザーが鳴ります
SW2 (赤) SW3 (緑) SW4 (青) SW6 (白)	押下	押下したスイッチと同色の二つの LED が点灯 SW2 赤→ LED7, LED3 点灯 SW3 緑→ LED6, LED2 点灯 SW4 青→ LED5, LED1 点灯 SW6 白→ LED4, LED0 点灯
VR1 (可変抵抗器)	回転	OLED の 4 行目に値 (0~255) が表示されています.
BME280		OLED の 2 行目に温度 (摂氏) が表示されています.
その他のチェックモード		
部品	方法	動作
SW5 (黄)	押下	MQTT 通信 SW5 押下でサーバに “MQTT_OUT” の文字列を送信 OLED ディスプレイの他の表示が消え、2 行目に “MQTT_OUT” と表示し、サーバとの通信成功時に OLED ディスプレイの 3 行目に “ok” が 2 秒程度表示 されます。その後、通常のチェックモードになります。
KEY1 (16 キーパッド)	押下	いずれかのキーを押下すると OLED ディスプレイの表示が切り替わり入力モードになります。 入力したキーを OLED ディスプレイに表示し、SW7 で 入力が終了になり、通常のチェックモードになります。
SW7 (黒)	押下	16 キーパッドの入力終了に使用

2 プログラム設計課題

2.1 課題内容

プログラム設計競技の課題数は、六つです。IoT ボードとマルチファンクションボードを接続して、ボード上の部品を仕様どおりに制御します。デジタル IO 制御、A/D 変換、OLED ディスプレイ制御、センサデータ処理、IoT 通信制御の内容を含み、仕様どおりに制御するプログラムを設計、記述、実装してください。

プログラム設計は、**公表 2**『3-5 プログラム設計競技仕様』、および以下に示す『プログラム設計仕様』に基づいて行ってください。また、ソースプログラムの記述方法は、**公表 2**『4-4 コーディング作法のガイドライン』にしたがってください。

(1) IoT ボード

マルチファンクションボードの四隅の穴に、図 2.1 に示すようにスペーサを取り付けてください。スペーサは、マルチファンクションボードの裏面側に取り付けます。締付けトルクは、セット小ねじのばね座金が完全につぶれ、スペーサが容易に手で回らない程度としてください。

荷札は、選手番号と氏名を記入の上、『3 提出仕様』にしたがって、IoT ボードの左下側に取り付けてください。

IoT ボードの電源を ON にすると、ESP32 マイコン上の LED が赤く点灯します。その後 ESP32 マイコンが Wi-Fi に接続されると、ブザーが「ピピッ」と鳴ります。鳴らない場合は、もう一度電源を入れ直して下さい。Wi-Fi に接続できない場合、MQTT 通信は動作しません。



図 2.1 スペーサの取付け方法

2.2 プログラム設計仕様

(1) 配付したソースファイルの構成

配付したソースファイルは，“iot_main.c”，“iot_xx.c”，および“iot_xx.h”の三つです．ソースファイルの構成を図 2.2 に示します．iot_main.c には，main 関数や各種初期設定，タイマ割込み処理などが記述されています．iot_xx.h には，define マクロや課題のプロトタイプ宣言が記述されています．iot_xx.c には，課題の処理関数が用意されていますので，各課題の仕様にしたがった処理関数のプログラムを記述してください．ただし，課題の処理関数以外の場所，例えばグローバル変数や新たな関数，割込み処理などを，iot_xx.c，iot_xx.h，iot_main.c 中に記述してもかまいません．

- iot_main.c: main()関数が入っているソースファイル（提出）
- iot_xx.c : 課題の処理関数が入っているソースファイル（提出）
- iot_xx.h : ヘッダファイル（提出）

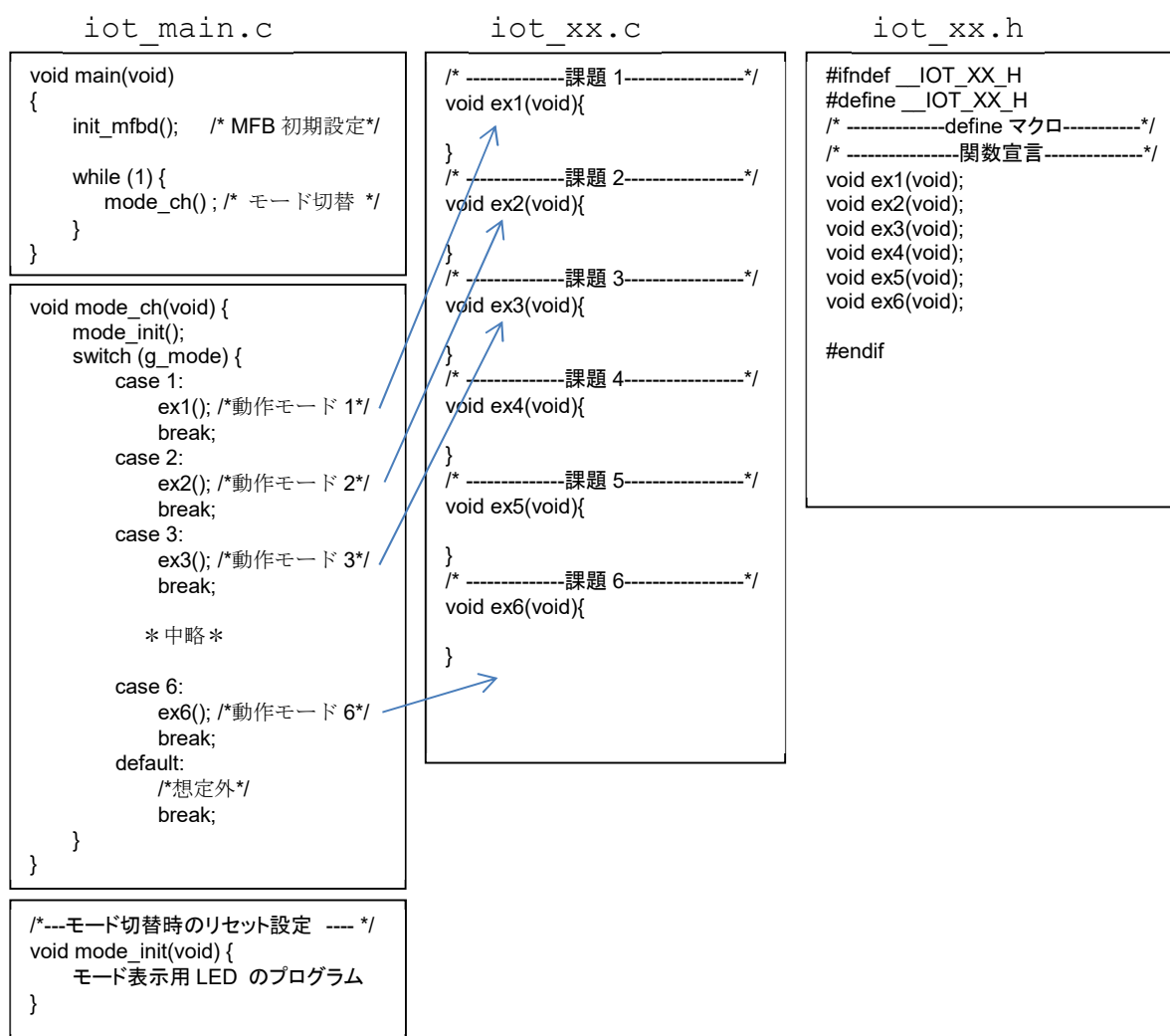


図 2.2 配付したソースファイルの構成

(2) 動作モード

以下の六つの動作モードのプログラムを設計、作成、実装することが課題です。

- モード1：タクトスイッチ操作による LED の点灯制御
- モード2：ボリューム操作による LED の点灯数制御
- モード3：OLED ディスプレイに XBM フォーマットの画像表示
- モード4：視覚復号型秘密分散の復号処理
- モード5：16 キーパッドによる入力データのサーバ送受信
- モード6：センサデータの取得とサーバへの送受信

動作モードの切り替えは、ロータリスイッチで行います。時計回りでモードの番号が進み、反時計回りで番号が戻ります。モード6の次はモード1に戻り、モード1を戻すとモード6になります。OLED ディスプレイの1行目に現在のモード番号が表示されます。モード切替に関するプログラムは変更しないでください。

- 電源投入時は、ESP32 マイコンがネットワークに接続されて“ピッ”と鳴ります。
- ロータリスイッチの LED は、電源投入時のネットワーク接続成功時のみ青く点灯しますが、その後ロータリスイッチの色が変わる場合がありますが問題はありません。
- 電源投入時は、モード1の `ex1()` が実行されます。
- モード1からモード6までのプログラムは、それぞれの処理関数 `ex1()` ～ `ex6()` に記述してください。ただし、タイマ1やその設定に関する部分などは、処理関数以外のところに記述してもかまいません。
- 処理関数 `ex1()` ～ `ex6()` の仮引数と戻り値は、`void` です。

(3) グローバル変数

表 2.1 に定義されている使用可能なグローバル変数を示します。これら以外のグローバル変数（システムグローバル変数）は、あらかじめ関数に使用している変数であるため使用しないでください。ローカル変数と区別するために、グローバル変数には変数名の頭文字に“g_”を付けています。

表 2.1 定義されているグローバル変数

グローバル変数	設定内容	割込み
<code>g_mode_flag</code>	モード切替検知用フラグ	—
<code>g_mqtt_flag</code>	MQTT データ取得確認用フラグ	—

- `g_mode_flag` は、モード切替があるとフラグが“1”になります。
- `g_mqtt_flag` は、課題6で使用します。サーバからデータが返ってくるとフラグが“1”になります。

(4) 初期設定

使用するポートの設定は、あらかじめ図 1.9 に示す入出力装置、A/D 変換、USART の構成に適合するように設定してあります。また、タイマ0は10msのインターバルタイマとしてディレイ関数に使用していますのでタイマの値は変えないでください。タイマ1については初期設定を行い、割込み処理の場所を設けてありますが、

ユーザ用に設けていますので変更して使用してもかまいません。

(5) 関数

課題で利用できる関数を表 2.2 に示します。必要に応じて使用してください。

表 2.2 定義されている関数

関数名	内容	使用例
<code>void delay_10ms(unsigned int)</code>	10ms 単位のディレイ 引数×10ms = ディレイ時間	1 秒ディレイ 例 <code>delay_10ms(100)</code>
<code>void wait_1ms(unsigned int value)</code>	1ms ディレイ 引数×1ms = ディレイ時間	5ms ディレイ 例 <code>wait_1ms(5)</code>
<code>void req_OLED_clear(void)</code>	OLED ディスプレイの画面クリア OLED ディスプレイは、クリアしない限り、画面はそのままです。	使用方法 ①画面のクリア ②表示したいデータをバッファにおくる ③実際に表示
<code>void req_OLED_buf(char *string, int col, int row)</code>	OLED ディスプレイのバッファ アルファベット、数字、記号は表示できます。 表示を変更したい場合は、まずバッファに送ってください。 引数①: string (char * 型) 表示したいデータ文字列とする 引数②: col (int 型) 表示したい列のドット番号 (0~127) 1 文字の幅は約 10 ドット 引数③: row (int 型) 表示したい行の番号 最大 4 行 (1~4)	例 画面の 2 行目に “TEST” を表示する <code>req_OLED_clear(); req_OLED_buf("TEST", 0, 2) req_OLED_display();</code>
<code>void req_OLED_display(void)</code>	OLED ディスプレイの表示 バッファ内のデータを表示します	
<code>int read_VR(void)</code>	ボリュームの情報取得 (A/D 変換値) 0~255	

<pre>float req_BME280(char *sensor);</pre>	センサ（温度，湿度，気圧）情報の取得 引数 sensor (char *型) 温度，湿度，気圧を指定する戻り値 (float 型)	温度を取得したいとき req_BME280("temp") 湿度を取得したいとき req_BME280("hum") 気圧を取得したいとき req_BME280("pres")
<pre>void publish_MQTT(char *topic, char *payload)</pre>	MQTT プロトコルのパブリッシュ（送信） 引数①: topic (char *型) トピック 文字列 引数②: payload (char *型) メッセージ 文字列	MQTT プロトコルでの送信トピックを "/kadai4" に設定し文字列 "a=7" をパブリッシュする <pre>publish_MQTT("/kadai4", "a=7")</pre>
<pre>char inputKey(void)</pre>	16 キーパッド (16KEY) の入力取得 戻り値 (char 型) 0～9, A～F	
<pre>void req_OLED_draw_graph(char * p_array, int wide, int height, int length_array, int g_x, int g_y)</pre>	配列 p_array に格納されている XBM データを OLED に表示 引数①: p_array (char *型) XBM データ 引数②: wide 表示画像の幅 (ドット数) 引数③: height 表示画像の高さ (ドット数) 引数④: length_array 配列の要素数 引数⑤: g_x 画像左上の x 座標 (0～127) 引数⑥: g_y 画像左上の y 座標 (0～63)	例: 64×32 の XBM フォーマット画像(image)を OLED 画面左上 (座標 (0,0)) に表示する場合 <pre>req_OLED_draw_graph(image, 64, 32, 256, 0, 0)</pre> OLED の座標は図 2.3 に示す

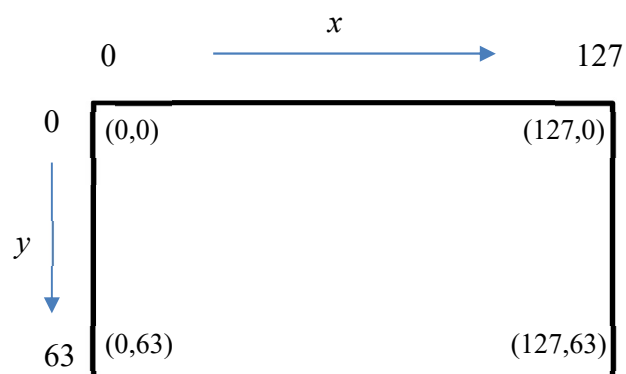


図 2.3 OLED ディスプレイの座標

2.3 プログラム設計課題

六つの課題があります。それぞれの課題では、提示した仕様以外の動作については問いません。

(1) 課題1：モード1 ex1(void)関数

十字キーの左右ボタン (SW2, SW4) を押下すると、点滅している LED が左右に移動する。

(仕様)

- モード切替時は LED4 が約 0.5 秒間隔で点滅していること。
- SW2 を押下すると、点滅している LED が左に一つ移動すること。
- SW4 を押下すると、点滅している LED が右に一つ移動すること。
- スイッチを押下した瞬間に移動すること。
- 1 回の押下で一つのみ移動すること。
- 左端 (LED7) では、SW2 を押下しても LED は移動しないこと。
- 右端 (LED0) では、SW4 を押下しても LED は移動しないこと。
- 押下した瞬間の状態は、点灯、消灯のどちらでもよい。

(2) 課題2：モード2 ex2(void)関数

ボリューム (VR1) の回転に合わせて LED (LED0～LED7) の点灯数を制御する。

(仕様)

- ボリューム (VR1) の回転位置に比例して LED の点灯数が全消灯から全点灯まで変化すること。
- VR1 を時計回りに回転させると、LED の点灯数が LED7 から LED0 まで順番に増加すること。
- VR1 を反時計回りに回転させると、点灯している LED が LED0 から LED7 の順番に減少すること。
- モード切替時は、VR1 の回転位置に応じた LED の点灯数とすること

(3) 課題3：モード3 ex3(void)関数

SW7 を押下すると図 2.4 に示す三角の図形データを XBM フォーマットで作成し、OLED ディスプレイに描画する。

(仕様)

- 図 2.4 に示す図形のデータを XBM 形式で作成し、128×64 ドット OLED ディスプレイに描画すること。
- 図 2.4 の白は白色、黒は黒色として OLED ディスプレイに表示すること。
- 配列データを関数で ESP マイコンに送るため表示に時間がかかるが、SW7 を押下後に 10 秒以内にすべての図形が描画されること。
- 図形に変形や欠けが無いこと。

- 描画中以外は，SW7 を再度押下すると，図形が再描画されること．
- モード切替時は，OLED ディスプレイの 1 行目に “mode 3” のみが表示された状態とすること．

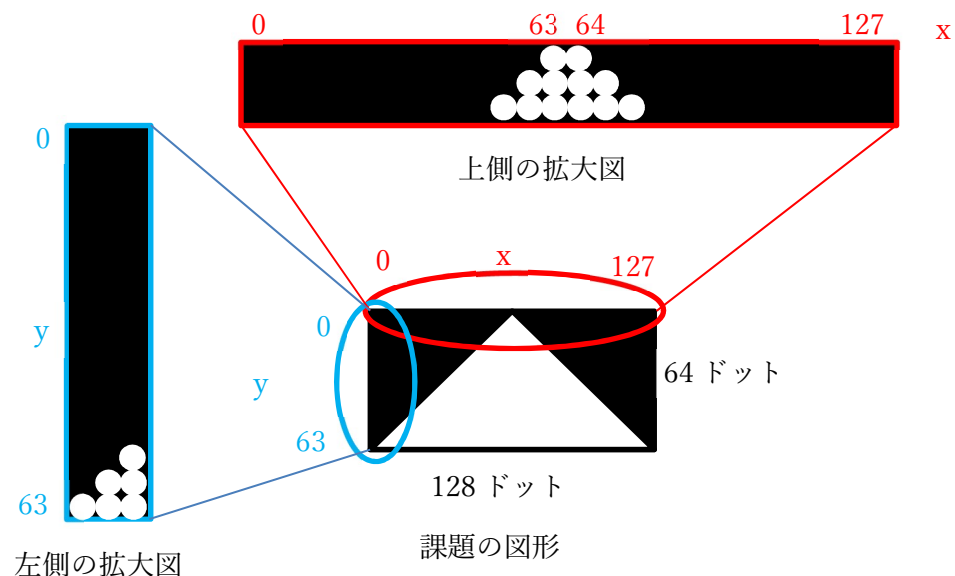


図 2.4 OLED に表示する三角画像

(4) 課題 4 : モード 4 ex4(void) 関数

2 枚に秘密分散された画像（暗号文）を 16 キーパッドの「A」または「B」の押下により，それぞれ OLED に表示させる．それらの画像を透明なシートに印刷したとして，2 枚を重ね合わせたときに表示（視覚復号）される画像を 16 キーパッドの「C」の押下により OLED に表示させる．

(仕様)

- 16 キーパッド (KEY1) の「A」を押下すると，秘密分散により暗号化された XBM 画像 A (iot_xx.h ファイルにある配列 pic_array1[1024]) を OLED に描画すること．
- KEY1 の「B」を押下すると，秘密分散により暗号化された XBM 画像 B (iot_xx.h ファイルにある配列 pic_array2[1024]) を OLED に描画すること．
- KEY1 の「C」を押下すると，「画像 A」，および「画像 B」の画像を透明なシートに印刷したとして，それらを復号処理として重ね合わせたときに表示される画像を OLED に描画すること．
- 描画終了後に KEY1 の「A」，「B」または「C」を押下すると，対応する画像が再描画されること．
- 配列データを関数で ESP マイコンに送るため表示に時間がかかるが，それぞれの画像は，KEY1 を押下してから 10 秒以内にすべてが描画されること．
- モード切替時は，OLED ディスプレイの 1 行目に “mode 4” のみが表示された状態とすること．

(5) 課題5：モード5 ex5(void)関数

16 キーパッドにより数値を MQTT プロトコルを用いてサーバに送信する。

(仕様)

- SW6 を押下すると 16 キーパッド (KEY1) の入力モードになり、OLED の 1 行目に “Enter data” と表示すること。
- KEY1 の任意のキーを押下すると OLED の 2 行目に押下した値が表示されること。
- KEY1 の任意のキーを続けて押下すると、複数桁の数値を入力でき、OLED の 2 行目に重ならないように表示されること。ただし、9 桁までの入力を確認できることとし、10 桁以上は表示されない部分があっても問わないこととする。
- SW7 を押下すると数値の入力が終了し、OLED の 3 行目に “Finish” と 1 秒間表示すること。このとき、入力した値をサーバに 1 回だけ MQTT 送信すること。ただし、送信するデータは、入力した値の先頭から最大 5 桁とする。その後、画面が消去され、OLED の 1 行目に “Send data” と表示し、2 行目にサーバに送信した数値を表示すること。
- MQTT プロトコル関数の topic には「/kadai5/key」、payload には入力した数値を適切に指定し、送信後 100ms 程度時間待ちを行うこと。
- 本課題では、受信の処理は記述する必要はありません。
- 送信が成功した場合、課題 4 の数値と一致または不一致について、送信した値に対する応答が OLED の 4 行目に自動的に文字列として表示されます。(ただし、課題 4 の完成未完成は本課題の採点に影響しません)
- SW6 を押下し上記の処理を繰り返せば、何度でも任意の桁数で任意に入力した数値を正確にサーバに送ることができること。
- モード切替時は、OLED ディスプレイの 1 行目に “mode 5” のみが表示された状態とすること。

(6) 課題6：モード6 ex6(void)関数

SW7 の押下により、温湿度センサから温度と湿度を読み込み、不快指数を計算し、それぞれを OLED ディスプレイに表示する。さらに、MQTT プロトコルを用いて不快指数をサーバに送信する。

- SW7 を押下すると、BME280 から温度と湿度情報を取得し、不快指数を計算し、OLED に表示すること。
- ディスプレイへの表示は、1 行目に “Temp = ” の後に摂氏の温度、2 行目に “Humi = ” の後に湿度、3 行目に “DI = ” の後に不快指数を、それぞれ小数点以下 2 桁までで表示すること。
- それぞれの数値の後に、摂氏の温度は “ C”，湿度は “ %” の単位を付けること。
- 不快指数は以下の計算式で求めること。

$$0.81T + 0.01H \times (0.99T - 14.3) + 46.3$$

ただし、T: 温度(°C) H: 湿度(%)

- SW7 の押下と連動して、OLED に表示した不快指数をサーバへ MQTT 送信すること。
- MQTT プロトコル関数の topic には「/kadai6」、payload には不快指数を適切に指定すること。
- サーバへの送信が成功すると、サーバから不快指数に対する応答が OLED の 4 行目に自動的に文字列として表示されます。
- OLED に表示した不快指数に対するサーバからの応答が表示されるまで不快指数を更新表示（変化）しないこと。
- サーバからの応答は 1 秒間表示させること。
- SW7 を押下し続けると、上記の処理である「温度、湿度、不快指数と、そのサーバからの応答」を 1 秒間隔でデータを更新表示させること。
- サーバから応答があると、g_mqtt_flag が“1”となります。g_mqtt_flag は、自動で“0”に戻らないため手動で“0”に戻すこと。
- センサの各データは、USART 通信による取得に失敗すると“-1200”が表示されますが、この動作は採点には影響しません。
- モード切替時は、OLED ディスプレイの 1 行目に“mode 6”のみが表示された状態とすること。

3 提出仕様

3.1 提出状態

IoT ボード、マルチファンクションボード、およびドキュメント類の提出状態については、**公表 2**『3-8 提出仕様（全競技に適用）』, および本章の『提出仕様』にしたがいなさい。

(1) IoT ボード、マルチファンクションボード

IoT ボード、およびマルチファンクションボードの提出状態は、表 3.1 に示すとおりとすること。

表 3.1 IoT ボード、マルチファンクションボードの提出状態

IoT ボード	
64 ピンコネクタ CN1	カップリングボードを介して、マルチファンクションボードと 接続 した状態
可変抵抗器 VR1	左に回しきった 状態
荷札	IoT ボードを正面から見て左下側のスペーサ部のセット ねじと一緒に荷札の針金を、締め込み 、容易に外れないように取り付けた状態
マルチファンクションボード	
PIC18F6722	課題プログラムが書き込まれている 状態
電源スイッチ SW1	オフ
RB6, RB7 ポート切り替えスイッチ SW3	バスライン側
RC6, RC7 ポート切り替えスイッチ SW4	バスライン側
DC ジャック CN1	何も挿入されていない状態 (電源アダプタは提出しない)
64 ピンコネクタ CN2	カップリングボードを介して、IoT ボードと接続した状態
機能拡張用コネクタ CN3, CN4	何も挿入されていない状態
モジュラジャック CN5	何も挿入されていない状態
USB コネクタ CN6	何も挿入されていない状態
SD メモリカードスロット CN7	何も挿入されていない状態
無線通信モジュールコネクタ J1, J2	何も挿入されていない状態
四隅の中空穴	配付した スペーサを取り付けた 状態

(2) プログラムソースリスト

提出するプログラムソースリストは、“iot_xx.c”, “iot_main.c”, およびヘッダファイル “iot_xx.h” である。なお、課題として作成した上記以外のプログラムソースリストがある場合は、併せて提出すること。

(3) 提出時の状態

提出する「課題作品」、「提出用封筒」、および「配付&回収用封筒」は、あらかじめ配付した用箋はさみの上に置くこと。

4 注意事項

4.1 質問

- (1) 質問があるときは、コールシステムの呼出しボタンを押し、競技委員に連絡しなさい。
- (2) 公表資料（ **公表 1** , **公表 2** ），職種連絡会資料，配付したドキュメント，資料に明確に記述されている事項，競技開始前に明確に説明した事項，および情報提供サーバに掲載されている事項に関する質問には回答できません。
- (3) 持込み機器（コンピュータ，測定器，電子機器など），および搭載ソフトウェア（Altium Designer, MPLAB X IDE, XC8 C コンパイラ，オシロスコープソフトウェア，ターミナルソフトウェアなど）の不具合，使用方法に関する質問には回答できません。
- (4) 質問に対する回答が即座にできない内容に関しては，競技委員間で協議の上，回答します。

4.2 資料，データシート

- (1) 別添資料 3 の「競技 III 部品表」のデータシート欄に「○」がついている部品，材料などの資料，データシートなどは，サーバ上の“datasheet3.zip”に PDF 形式で保存されています。これらの資料，データシートなどは必要に応じて，使用してください。